SkillOps: A Practical Framework for Designing, Testing, and Operating Modular Skills in Personal AI Agents

Song Luo
luosongred@gmail.com

June 2026

Abstract

Personal AI agents increasingly rely on reusable capability modules, often called skills, yet their design and operation remain largely informal. This paper introduces SkillOps, a practical framework for designing, testing, and operating modular skills in personal AI agents. The framework is derived from open-source artifacts developed by the author and contributes a skill component taxonomy, a lifecycle model, a failure-mode taxonomy, a design-gap matrix, and an artifact-based evaluation design. The paper reports completed descriptive summaries, repository-level consistency checks, a local security-guard pilot, and live model-backed runs with DeepSeek and Kimi over trigger, constraint, security, and memory-drift protocols. It also reports an external-validation readiness layer: a metadata-only third-party corpus protocol, a seeded 24-artifact pilot plan, bounded runner readiness, and a two-annotator calibration worklist. The contribution is therefore a bounded engineering framework for making agent skills more explicit, testable, and maintainable, not a claim that a particular skill format universally improves model behavior.

1 Introduction

Large language model agents are increasingly used as interactive systems that can read instructions, call tools, edit files, use external APIs, and maintain task-specific context. In such systems, user-visible behavior is shaped not only by the base model, but also by auxiliary capability modules. These modules may be called tools, plugins, agents, memories, workflows, or skills. This paper focuses on the skill as a practical unit of agent capability in personal agent environments.

In many personal agent settings, skills are written and maintained by individual users or small teams rather than by centralized platform engineers. A skill may contain a trigger description, operational instructions, examples, constraints, memory rules, safety checks, and tests. When these elements are implicit or poorly specified, the agent may activate the wrong skill, inject excessive context, ignore constraints, retain obsolete information, or perform unsafe operations. These are not only model-quality issues; they are operational issues in the design, testing, and maintenance of modular agent capabilities.

This paper introduces SkillOps, a practical framework for the lifecycle management of agent skills. The term is used by analogy to operational practice in software engineering, but the scope is narrower. SkillOps does not propose a new model architecture or a general theory of agents. Instead, it provides a structured way to define, test, audit, and revise skills as reusable capability units.

The framework is derived from five open-source artifacts:

- `skill-design-guide`: guidance for defining skill structure and usage boundaries.
- `skill-security-guard`: checks for unsafe or ambiguous skill behavior.
- `persistent-memory`: a file-based memory mechanism for continuity across agent sessions.
- `agent-self-audit`: a self-auditing mechanism for reviewing agent outputs and operational risks.
- `lobster-guard`: a guardrail-oriented tool for checking operational assumptions and failure modes.

The central claim is operational rather than architectural: personal agent skills become easier to inspect, test, and govern when they are treated as lifecycle-managed artifacts with explicit structure, tests, and operational controls. The paper contributes a framework, a taxonomy of skill components, a methodology for artifact-based study, a descriptive benchmark layer built from manually constructed inputs, a local executable risk-review pilot, and a third-party corpus protocol with pilot-readiness artifacts. The reported tables, pilots, and readiness files should not be read as broad empirical validation or as a general model comparison.

2 Background and Motivation

2.1 From Prompts to Operational Skills

Prompt engineering is often discussed as the design of instructions for a language model. In personal agent systems, however, instructions are rarely isolated. They may be tied to tools, files, context windows, memory policies, or task-specific workflows. A skill can therefore be understood as a structured capability unit that combines instructions with operational assumptions.

For example, a writing skill may define when it should be used, what tone it should adopt, what sources it may rely on, and how it should handle uncertainty. A code-review skill may specify repository access assumptions, test execution rules, and failure-reporting requirements. A memory skill may specify what information should be retained, where it should be stored, and how obsolete information should be forgotten. These examples suggest that a skill is closer to an operational module than to a single prompt.

2.2 Operational Failure Modes

Informal skills can fail in several recurring ways. A skill may be triggered too broadly or too narrowly. It may inject irrelevant or stale context, causing the model to overfit to past assumptions. It may contain constraints that are underspecified or not testable. It may introduce safety risks when it authorizes file edits, command execution, credential handling, or external communication. It may also accumulate obsolete memory that conflicts with current user intent.

These failure modes motivate a more systematic approach. If skills are treated as artifacts, they can be linted, tested, reviewed, versioned, and retired. This does not eliminate model uncertainty, but it can make operational assumptions more visible and auditable. Table 1 summarizes the recurring failure categories used throughout the framework.

Failure mode	Description	Typical mitigation
Over-triggering	The skill activates for requests that only superficially match its scope.	Narrower trigger language, explicit negative examples, boundary tests.
Under-triggering	The skill fails to activate for valid requests.	Positive examples, paraphrase coverage, and missed-activation checks.
Context contamination	Irrelevant, stale, or conflicting context is injected into execution.	Source filtering, freshness checks, conflict-resolution rules.
Constraint drift	The skill's constraints are vague, contradictory, or not enforced consistently.	Testable rules, lint checks, regression cases.
Unsafe tool use	The skill authorizes risky file, command, network, or credential operations without sufficient guardrails.	Permission scoping, security scanning, post-run audit questions.
Memory drift	Stored memory persists after the underlying assumption has changed.	Expiry rules, deletion paths, explicit retirement and forgetting policies.
Audit blindness	Failures are not surfaced because the skill does not require self-review or evidence reporting.	Output contracts, self-audit prompts, artifact logging.

Table 1: Failure-mode taxonomy for skill-based personal agents.

2.3 Positioning of SkillOps

SkillOps is positioned between prompt design, tool-use agents, and software engineering practice. It does not attempt to replace agent architectures such as tool-use planning, reasoning-action loops, or autonomous software engineering agents. Instead, it addresses a complementary layer: how reusable skills should be specified and operated in a personal agent environment.

Prior work on tool-using agents studies how language models decide when and how to use external tools [38, 48]. Work on agent-computer interfaces studies how environment design affects agent performance [47]. Work on reusable skill libraries shows that agent capabilities can be stored and reused over time [45]. SkillOps builds on this general direction but focuses on the practical lifecycle of user-defined skills: design, triggering, context injection, constraint enforcement, auditing, and forgetting.

3 The SkillOps Framework

3.1 Definition of a Skill

In this paper, a skill is defined as a reusable capability unit that guides an agent's behavior for a class of tasks. A skill may include natural-language instructions, examples, trigger conditions, context requirements, tool permissions, safety constraints, output contracts, and tests.

SkillOps-managed skills are intended to document these assumptions in a more structured and reviewable form. Table 2 summarizes the structural components used in the framework, and Figure 1 provides a schematic view of how those components fit together in a skill package.

3.2 Lifecycle Model

SkillOps organizes skill management into a lifecycle:

1. **Design**: define purpose, scope, triggers, constraints, and output expectations.

Component	Operational role	Example question
Name and purpose	Defines the task class and intended value of the skill.	What problem is this skill meant to solve?
Trigger boundary	States when the skill should and should not activate.	What requests are in scope, out of scope, or ambiguous?
Input assumptions	Records required user or environment information.	What must be known before execution starts?
Context policy	Specifies what prior context, files, or memory may be injected.	What supporting context is necessary, and how fresh must it be?
Execution constraints	States required, optional, and forbidden actions.	What must the agent do, avoid, or verify?
Tool and file permissions	Limits resource access during execution.	Which tools, files, networks, or services are allowed?
Output contract	Defines the expected form of the answer or artifact.	What must the final output contain?
Safety and security checks	Surfaces risks and policy-relevant conditions.	What unsafe actions, hidden assumptions, or trust-boundary issues must be checked?
Tests and examples	Provides positive, negative, and edge cases.	How is the skill regression-tested?
Lifecycle metadata	Records versioning, revision triggers, and retirement rules.	When should the skill be updated, deprecated, or forgotten?

Table 2: Structural components of a SkillOps-managed skill.

2. **Lint**: check for ambiguity, missing fields, conflicting instructions, unsafe permissions, and untestable claims.
3. **Test**: evaluate the skill on positive, negative, boundary, and adversarial examples.
4. **Deploy**: make the skill available to the agent under controlled activation conditions.
5. **Observe**: collect failures, unexpected activations, user corrections, and context conflicts.
6. **Audit**: review outputs and operational traces for risk, drift, and non-compliance.
7. **Revise**: update the skill based on observed failures.
8. **Forget or retire**: remove obsolete context, deprecated rules, or unsafe skills.

The lifecycle emphasizes that a skill is not complete when it is written. Its behavior depends on activation, context, model behavior, and interaction with other skills. Figure 2 summarizes the lifecycle stages from design through forgetting or retirement.

3.3 Trigger Design

Trigger descriptions determine when a skill is activated. A broad trigger may widen activation coverage but also cause irrelevant activation. A narrow trigger may reduce false positives but fail to activate in valid cases. SkillOps therefore recommends that trigger descriptions include both positive and negative examples.

A trigger specification should answer three questions:

- What user requests should activate the skill?

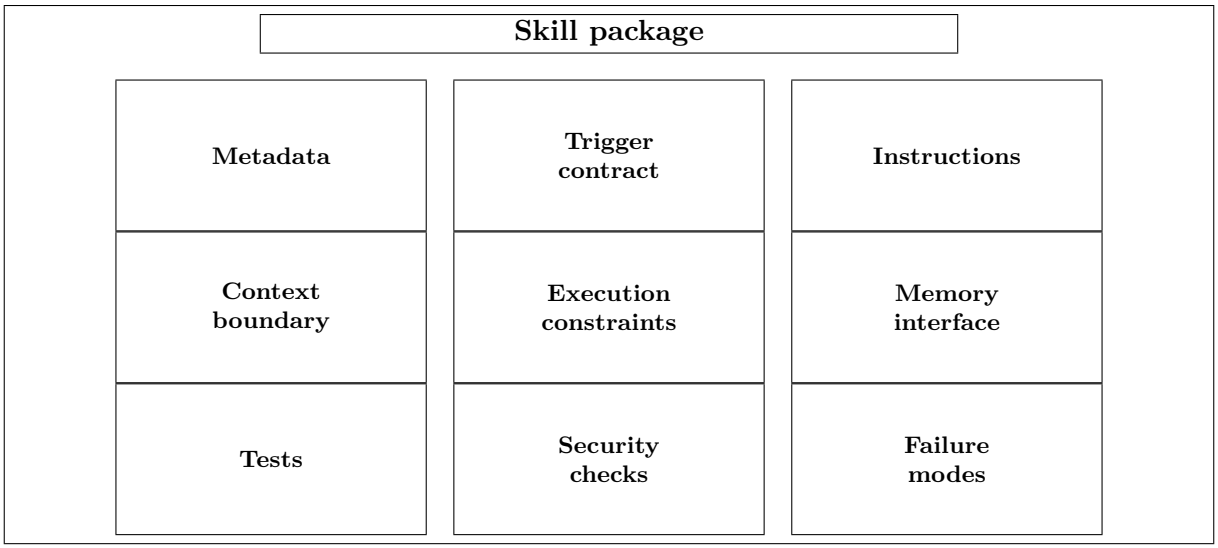

Figure 1: LaTeX-native rendering of a SkillOps skill package, with the package boundary enclosing metadata, trigger contract, instructions, context boundary, execution constraints, memory interface, tests, security checks, and failure modes.

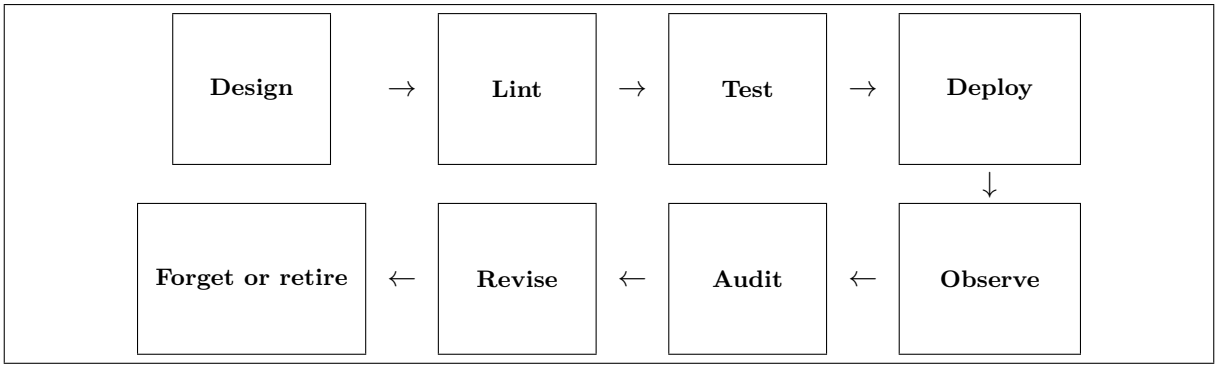

Figure 2: LaTeX-native rendering of the SkillOps lifecycle, showing the linked sequence of design, lint, test, deploy, observe, audit, revise, and forgetting or retirement.

- What superficially similar requests should not activate it?
- What ambiguity should cause the agent to ask for clarification or fall back to a general response?

Trigger quality can be evaluated through paraphrase tests, boundary tests, and multi-intent tests. This paper reports only the descriptive composition of a manually constructed trigger benchmark; Section 6 summarizes the reported counts and their limitations.

3.4 Context Injection

Context injection is necessary when a skill depends on project conventions, user preferences, prior decisions, or repository structure. However, excessive context can degrade behavior by introducing stale assumptions or irrelevant constraints. SkillOps treats context injection as a declared policy rather than opportunistic accumulation of memory.

A context injection policy should specify:

- the minimum context needed for the skill to operate;
- the source of the context, such as files, memory entries, or tool outputs;
- the freshness requirements for the context;
- the conditions under which context should be omitted;
- the method for resolving conflicts between current user instructions and stored context.

This is especially important for personal agents, where long-lived memory can improve continuity but also preserve outdated assumptions.

3.5 Execution Constraints

Execution constraints define what the agent is allowed to do during skill use. For low-risk writing skills, constraints may concern tone, citation style, or formatting. For higher-risk skills, constraints may concern file modification, command execution, external communication, credential handling, or irreversible actions.

SkillOps distinguishes between three types of constraints:

- **Behavioral constraints**: rules about tone, reasoning style, response structure, or uncertainty reporting.
- **Operational constraints**: rules about tools, files, commands, commits, network access, and external services.
- **Safety constraints**: rules about privacy, credentials, harmful actions, or irreversible operations.

A constraint is more useful when it is testable. For example, "do not claim success without evidence" is more testable than "be careful." SkillOps therefore favors constraints that can be checked through examples, lint rules, or audit questions.

3.6 Forgetting and Retirement

Forgetting is a necessary part of skill operation. A personal agent may retain preferences, project state, or procedural rules that later become obsolete. Without a forgetting mechanism, the agent may apply outdated instructions to new tasks.

SkillOps treats forgetting at two levels. First, individual memory entries may need to be removed or revised. Second, entire skills may need to be deprecated when their scope changes or when they become unsafe. Retirement rules should specify how to identify obsolete skills, how to preserve historical context if needed, and how to prevent deprecated instructions from continuing to influence behavior.

Table 3 contrasts informal prompt-based skills with skills managed under the proposed framework.

Dimension	Informal skill	SkillOps-managed skill
Scope definition	Implicit task description embedded in prompt text.	Named purpose, explicit scope, and documented non-scope.
Activation	Heuristic or ad hoc triggering.	Trigger contract with positive, negative, and boundary examples.
Context handling	Opportunistic context accumulation.	Declared context sources, freshness rules, and conflict handling.
Constraints	Broad instructions such as "be careful."	Testable behavioral, operational, and safety constraints.
Testing	Rare or manual spot checks.	Regression cases, adversarial examples, and lint coverage.
Security review	Often absent or implicit.	Permission scoping, unsafe-action checks, and trust-boundary review.
Memory handling	Persistent context without explicit expiry rules.	Defined write permissions, forgetting policy, and retirement criteria.
Operational reporting	Minimal trace of what changed and why.	Output contract, audit fields, versioning, and revision history.

Table 3: Comparison between informal prompt-based skills and SkillOps-managed skills.

Axis	Informal handling	SkillOps interface
Activation	Broad task description or keyword matching.	Trigger contract with positive, negative, and ambiguous cases.
Context	Whatever prior state is convenient to include.	Declared sources, freshness rules, and conflict handling.
Constraints	General cautionary language.	Testable behavioral, operational, and safety constraints.
Memory	Persistent context without retirement semantics.	Explicit write, read, forgetting, and deprecation policy.
Audit	Success or failure reported after the fact.	Output contract, evidence fields, and review checks tied to known failure modes.

Table 4: SkillOps turns coupled skill-design risks into explicit interfaces that can be reviewed and regression-tested.

3.7 Problem Formulation and Design Gap

The core problem can be stated as follows. Given a set of reusable skills $\mathcal{S}$, a user request x, optional context c, and an execution environment e, an agent must decide whether to activate a skill, which context to expose, which operations to allow, and how to report uncertainty or failure. Informal prompt-based skills usually compress these decisions into a single natural-language instruction. SkillOps separates them into reviewable interfaces: activation, context, constraints, memory, and audit.

This separation matters because the failure modes are coupled. An over-broad trigger can expose irrelevant memory; stale memory can defeat otherwise clear constraints; weak output contracts can hide unsafe tool use; and missing retirement rules can keep obsolete behavior alive after a project changes. Table 4 summarizes the design gap addressed by the framework.

4 Research Questions

This paper is organized around three research questions.

Research question	Framework component	Evaluation method
RQ1	Skill structure and documentation fields	Artifact review, template comparison, and completeness analysis across the open-source repositories.
RQ2	Trigger design, context policy, execution constraints, and forgetting	Manually constructed benchmark cases and model-backed protocols covering activation boundaries, stale-context injections, and deprecated-memory scenarios.
RQ3	Linting, security scanning, and self-audit	Seeded failure cases, reviewable repository checks, and manual review criteria for operational risk inspection.

Table 5: Mapping from research questions to framework components and evaluation methods.

RQ1: What structural components should a skill include as an agent capability unit? This question examines whether a practical skill template can make agent capabilities easier to understand, test, and maintain. The expected output is a component taxonomy covering triggers, context, constraints, tool access, output contracts, tests, and lifecycle metadata.

RQ2: How do trigger descriptions, context injection, execution constraints, and forgetting expose operational risk? This question examines how design choices influence activation behavior, consistency across paraphrases, constraint following, and resistance to stale context. The expected output is a benchmark suite and a provider-neutral experimental protocol with controlled variations of skill definitions.

RQ3: How can linting, security scanning, and self-auditing support operational risk review through reviewable checks? This question examines how reviewable checks can surface common skill risks and make them easier to inspect. The expected output is a manually constructed set of lint, security, and self-audit cases together with repository-level checks and review criteria for operational risk review. It does not by itself prove that these mechanisms reduce deployment risk or improve model behavior.

Table 5 maps each research question to the corresponding framework component and evaluation method.

5 Artifacts and Methodology

5.1 Artifact Base

The framework is derived from a set of open-source artifacts. Table 6 summarizes their intended roles.

The repository location for the paper and its supporting materials is provided in Section 5.4.

5.2 Methodological Approach

This paper uses an artifact-based methodology. Rather than beginning with a large-scale user study, it derives a framework from the design and use of practical open-source tools, then tests parts of the framework through repository checks and live model calls. The methodology has six steps:

Artifact	Role in SkillOps
`skill-design-guide`	Provides design guidance for structuring skills, defining scope, and writing trigger descriptions.
`skill-security-guard`	Provides checks for unsafe, ambiguous, or overly permissive skill behavior.
`persistent-memory`	Provides a file-based approach to retaining and distilling long-term agent context.
`agent-self-audit`	Provides a mechanism for reviewing agent outputs, assumptions, and possible failure modes.
`lobster-guard`	Provides operational guardrails for checking assumptions, reporting partial failure, and avoiding unsupported claims.

Table 6: Open-source artifacts used as the practical basis for the SkillOps framework.

1. **Artifact review**: identify recurring structures and constraints across the repositories.
2. **Component abstraction**: generalize these structures into a skill component taxonomy.
3. **Failure-mode analysis**: identify common ways in which skills can misfire, become stale, or create operational risk.
4. **Design-gap mapping**: map each failure mode to the skill interface that should make it inspectable.
5. **Benchmark design**: construct benchmark cases intended for later evaluation of trigger behavior, context injection, constraint following, forgetting, linting, security scanning, and self-audit behavior.
6. **Model-backed execution**: run provider-neutral prompts over trigger, constraint, security, and memory-drift cases with preserved raw logs and recomputable metrics.
7. **External-corpus planning**: define a third-party artifact sampling frame and validation protocol before making broad ecosystem claims.

This approach is exploratory. It is appropriate for deriving a framework, but it cannot by itself establish broad generality. The limitations are discussed in Section 8.

5.3 Benchmark Construction and Descriptive Outputs

The benchmark includes manually constructed cases in the following categories:

- **Activation cases**: requests that should activate a skill.
- **Non-activation cases**: requests that appear related but should not activate the skill.
- **Boundary cases**: ambiguous requests requiring clarification or fallback.
- **Context cases**: tasks requiring relevant context and tasks where injected context is harmful.
- **Constraint cases**: tasks testing whether operational and safety constraints are followed.
- **Forgetting cases**: tasks where obsolete memory or deprecated instructions should not be used.
- **Security cases**: tasks containing unsafe permissions, credential exposure, prompt injection, or irreversible actions.

- **Audit cases**: completed outputs requiring self-review for unsupported claims, missing evidence, or hidden assumptions.

The repository includes manually constructed benchmark inputs and generated descriptive outputs. `benchmark/skill_samples.csv` profiles five artifacts, `benchmark/trigger_cases.csv` contains 36 trigger cases, and `benchmark/risk_cases.csv` contains 24 operational-risk cases. Running `scripts/run_all.py` regenerates summary tables under `results/tables/` and SVG figures under `figures/`. Because the benchmark is manually constructed and exploratory, these outputs should be read as descriptive summaries rather than as statistical evidence or broad empirical validation. The repository also includes executable repository-level consistency checks that verify benchmark schema, result reproducibility, paper-claim consistency, public-presentation constraints, SVG validity, and evidence-matrix integrity. Live model-backed runs are stored under `results/experiments/raw/` and summarized by `scripts/summarize_live_model_results.py`. These runs score the provided prompts and case sets; they do not execute the five source repositories as deployed software systems.

5.4 Artifact Availability

The public repository for this paper is `https://github.com/rrrrrredy/skillops-paper`. The versioned artifact release used for this revision is `v1.2.0` with Zenodo version DOI 10.5281/zenodo.20900771; the Zenodo concept DOI for the release family is 10.5281/zenodo.20061198 [24].

Supporting materials are organized as follows:

- Artifact inventory: `artifacts/artifact_inventory.md`
- Benchmark input: `benchmark/skill_samples.csv`
- Benchmark input: `benchmark/trigger_cases.csv`
- Benchmark input: `benchmark/risk_cases.csv`
- Reproducible scripts: `scripts/`
- Preserved live-run outputs and metrics: `results/experiments/`
- External artifact sampling frame: `benchmark/external_artifact_corpus_sources.csv`
- External validation protocol: `experiments/external_validation_protocol.md`
- Generated result tables: `results/tables/`
- Generated figure artwork: `figures/`

These materials are intended to make the exploratory benchmark inspectable, but the benchmark remains manually constructed and should not be interpreted as broad empirical validation.

6 Evaluation

This revision reports four evidence layers currently versioned in the repository. The first is a descriptive layer derived from manually constructed benchmark artifacts. The second is a deterministic repository-check layer. The third is a local security-guard pilot over seeded risk cases and benign controls. The fourth is a model-backed live-run layer over DeepSeek `deepseek-v4-flash`

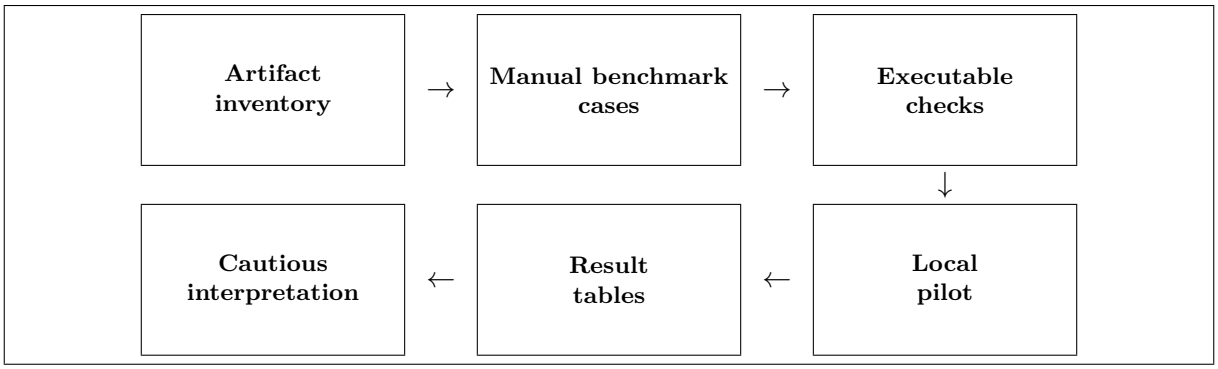

Figure 3: Evaluation pipeline from artifact inventory and manual benchmark cases through executable checks, result tables, a local pilot, and cautious interpretation.

and Kimi `kimi-k2.7-code`. No statistical significance is claimed, no broad empirical validation is claimed, and the results should be read as evidence of inspectability and executable review rather than proof that SkillOps is generally effective.

6.1 Evaluation Scope

The evaluation combines the following versioned inputs:

- Skill samples: `benchmark/skill_samples.csv`
- Trigger cases: `benchmark/trigger_cases.csv`
- Risk cases: `benchmark/risk_cases.csv`
- Benign controls: `experiments/security_benign_cases.csv`
- Cross-checking notes: `artifacts/artifact_inventory.md`
- Local pilot metrics: `results/experiments/security_guard_metrics.csv`
- Live model summary: `results/experiments/live_model_summary.csv`
- External corpus frame: `benchmark/external_artifact_corpus_sources.csv`

These inputs cover five artifact profiles, 36 trigger-routing cases, 24 operational-risk cases, 24 benign security controls, 22 memory-drift cases, and 11 third-party corpus sources. Ten GitHub-hosted third-party sources are also analyzed with a metadata-only file-tree pass. All trigger and risk cases use the `manual-` prefix, making the manual construction of the benchmark explicit. Figure 3 captures the evaluation flow from artifact inventory to cautious interpretation.

6.2 Descriptive Benchmark Summaries

The descriptive benchmark layer documents benchmark composition and artifact coding only. Running `scripts/run_all.py` regenerates summary tables under `results/tables/` and SVG figures under `figures/`, but it does not execute the five source repositories or score routing, linting, scanning, or self-audit behavior.

Component	Documented	Limited	Absent
Metadata	5	0	0
Trigger contract	5	0	0
Instructions	5	0	0
Context boundary	5	0	0
Execution constraints	5	0	0
Memory interface	1	4	0
Tests	3	2	0
Security checks	5	0	0
Failure modes	5	0	0

Table 7: Descriptive artifact-coverage counts generated from the manually constructed artifact profiles.

Dimension	Category	Count
Expected label	`should_trigger`	15
Expected label	`should_not_trigger`	12
Expected label	`ambiguous`	9
Relevant skill	`skill-design-guide`	7
Relevant skill	`skill-security-guard`	6
Relevant skill	`persistent-memory`	7
Relevant skill	`agent-self-audit`	4
Relevant skill	`lobster-guard`	5
Relevant skill	`none`	7

Table 8: Descriptive trigger-case counts generated from the manually constructed trigger benchmark.

Artifact coverage. Table 7 summarizes the descriptive coding of nine SkillOps-relevant components across the five inspected artifacts. Metadata, trigger contracts, instructions, context boundaries, execution constraints, security checks, and failure modes are documented in all five artifacts. Memory interfaces are documented in one artifact and limited in four, while tests are documented in three artifacts and limited in two.

Trigger cases. The trigger benchmark contains 36 manually written cases: 15 positive activation cases, 12 negative cases, and 9 ambiguous cases. The cases are distributed across the five inspected skills plus seven no-route cases that are intended not to activate any inspected skill. Table 8 reports these counts directly from the generated summary files.

Risk cases. The risk benchmark contains 24 manually written cases. The case inventory is evenly split across eight risk types, with three cases each for prompt injection, over-broad triggers, unsafe file access, missing constraints, stale memory, missing tests, identity confusion, and privacy leakage. Table 9 also reports the artifact emphasis of the case set, with six cases tied most directly to `persistent-memory`, five each to `skill-security-guard` and `lobster-guard`, and four each to `skill-design-guide` and `agent-self-audit`.

6.3 Repository-Level Consistency Checks

The repository includes `scripts/run_tests.py`, a deterministic repository-level test suite for consistency and traceability checks. In the current repository state, the suite covers benchmark schema,

Dimension	Category	Count
Risk type	`prompt_injection`	3
Risk type	`over_broad_trigger`	3
Risk type	`unsafe_file_access`	3
Risk type	`missing_constraints`	3
Risk type	`stale_memory`	3
Risk type	`missing_tests`	3
Risk type	`identity_confusion`	3
Risk type	`privacy_leakage`	3
Relevant artifact	`skill-security-guard`	5
Relevant artifact	`lobster-guard`	5
Relevant artifact	`skill-design-guide`	4
Relevant artifact	`persistent-memory`	6
Relevant artifact	`agent-self-audit`	4

Table 9: Descriptive risk-case counts generated from the manually constructed operational-risk benchmark.

Metric	Value	Count
Risk detection rate	1.000000	24/24
Benign false-positive rate	0.041667	1/24
Benign specificity	0.958333	23/24

Table 10: The local security-guard pilot caught all seeded risk cases and flagged one benign control, showing executable risk-review plumbing rather than broad security validation.

result reproducibility, paper-claim consistency, public-presentation constraints, SVG validity, and evidence-matrix integrity, and it passes the full repository test suite.

These checks validate repository consistency and traceability only. They do not measure model behavior, scanner accuracy, user-study outcomes, production validation, or statistical significance.

6.4 Local Security-Guard Pilot

The local pilot executes `scripts/run_security_guard_experiment.py` with the `local-rules` guard over the 24 risk cases and 24 benign controls. It is intentionally simple: the guard is a deterministic pattern-based checker, so the pilot evaluates whether the seeded risk corpus and benign controls are wired into an executable review loop, not whether a learned model can identify new risks.

All eight risk categories had three of three seeded cases detected in this local run. The single benign false positive is useful rather than incidental: it indicates that guard checks need benign controls whenever the risk detector uses broad textual patterns.

6.5 Model-Backed Live Runs

The repository includes provider-neutral live-run protocols for trigger routing, constraint handling, security review, memory drift, and skill-component ablations. The protocol fixes input files, prompt templates, JSON schemas, metric calculations, and dry-run validation before any live call. For this revision, the four core protocols were run once with DeepSeek `deepseek-v4-flash` and Kimi `kimi-k2.7-code` through OpenAI-compatible chat-completion APIs [10, 28]. Raw JSONL outputs are preserved under `results/experiments/raw/`; the cross-model summary is recomputed with `scripts/summarize_live_model_results.py`.

Model	Catalog	F1	False trigger	Ambiguity handled
DeepSeek v4 Flash	SkillOps	0.857143	0/12	4/9
DeepSeek v4 Flash	Freeform	0.833333	0/12	3/9
Kimi K2.7 Code	SkillOps	0.882353	0/12	4/9
Kimi K2.7 Code	Freeform	0.909091	0/12	6/9

Table 11: Trigger-routing live-run metrics over 36 manually constructed cases. The results are single-run, benchmark-specific metrics rather than statistical evidence of general superiority.

Model	Condition	Constraint	Forgetting	Conflict
DeepSeek v4 Flash	SkillOps/full policy	7/24	22/22	21/22
DeepSeek v4 Flash	Vague/no forgetting	3/24	0/22	2/22
Kimi K2.7 Code	SkillOps/full policy	11/24	22/22	21/22
Kimi K2.7 Code	Vague/no forgetting	12/24	0/22	1/22

Table 12: Constraint and memory live-run metrics. Constraint compliance is not monotonic across models, while explicit forgetting is robust in this small benchmark.

Table 11 reports trigger-routing results. In this small case set, both models avoided false triggers on all should-not-trigger cases. The relative ordering between the structured SkillOps catalog and the freeform catalog is model-dependent: DeepSeek favors the structured catalog on F1 and ambiguity handling, while Kimi favors the freeform catalog. This is a useful negative result for the paper's central claim: the framework should not be read as a universal prompt-performance improvement, but as an operational interface for making routing behavior inspectable.

Table 12 reports constraint and memory results. The constraint task is intentionally hard: the model must identify operational risk and avoid unsupported success claims from short scenarios. SkillOps improves DeepSeek's constraint compliance over the vague prompt in this run, but not Kimi's. By contrast, the explicit memory policy is stable across both models: neither uses stale information in the full-policy condition, both apply the forgetting rule in all 22 cases, and both resolve 21 of 22 conflict cases.

Table 13 reports the model-backed security-guard run. Kimi detects 23 of 24 seeded risk cases and DeepSeek detects 22 of 24; both allow all 24 benign controls. These results support the feasibility of a model-backed guard protocol with benign controls, but the seeded corpus remains too small for broad security claims.

6.6 External Corpus and User-Study Protocol

The repository now includes an external sampling frame rather than relying only on the author's artifact ecosystem. The frame covers the Agent Skills specification and public skills repositories, MCP reference servers and high-permission server examples, OpenAI Agents SDK examples, AutoGen and LangGraph workflow templates, Open WebUI function recipes, and a prompt-only baseline [1, 4, 27, 29, 30, 46]. These sources are recorded in `benchmark/external_artifact_corpus_sources.csv`; the corresponding model-backed and human-review protocol is recorded in `experiments/external_validation_protocol.md`. The protocol is now backed by an executable case scaffold: `experiments/schemas/external_case_schema.json` defines external case records, `experiments/external_case_seed.csv` provides seed cases, `scripts/generate_external_case_plan.py` deterministically writes `results/tables/external_case_allocation.csv`, `results/tables/external_case_plan.csv`, and `results/tables/external_condition_plan.csv`, and `docs/annotation_guide.md` plus `docs/preregistration_template.md` specify labeling

Model	Risk detection	False positives	Specificity
DeepSeek v4 Flash	22/24	0/24	24/24
Kimi K2.7 Code	23/24	0/24	24/24

Table 13: Model-backed security-guard live-run metrics over 24 seeded risk cases and 24 benign controls.

and pre-analysis rules. The scaffold assigns 240 target artifact slots to source strata, yielding 960 planned base cases and 2880 planned condition rows; these files are protocol artifacts, not outcome data.

A second metadata-only layer selects candidate artifact references for that allocation. `scripts/select_external_artifacts.py` reads the allocation and public Git trees, then writes `results/tables/external_artifact_selection.csv` and `results/tables/external_artifact_selection.md`. It records source versions, repository paths or upstream links, and selection bases without copying third-party prose or code. The resulting inventory contains 232 concrete metadata-only third-party references plus 8 pending replacement slots within the 240-slot design: 49 `SKILL.md`-anchored directories, 20 index-derived upstream links, 24 manifest directories, 15 README-scoped directories, 106 relevant tree paths, 18 additional text-like tree paths, and 8 replacement slots. These rows make the external study auditable at the artifact-reference level, but they still require replacement, eligibility review, and annotation before behavioral claims can be made.

The repository also contains a planned annotation and execution packet derived from those references. `scripts/generate_external_annotation_packet.py` writes `results/tables/external_case_construction.csv`, `results/tables/external_annotation_packet.csv`, and `results/tables/external_condition_packet.csv`. The construction table contains four protocol-seeded base cases per candidate artifact: one positive trigger, one negative trigger, one boundary-clarification case, and one risk-or-constraint case. The annotation packet keeps two independent reviewer fields and separate adjudication fields for all 960 base cases; the condition packet crosses those cases with original/freeform, SkillOps-normalized, and SkillOps-ablation representations, yielding 2880 pending condition rows. All rows are marked as review or construction work, not as completed annotation or execution evidence.

Finally, `scripts/run_external_condition_dry_run.py` validates the 2880 pending condition rows and writes `results/experiments/external_condition_manifest.csv`, `results/experiments/external_condition_shards.csv`, and `results/experiments/external_statistical_analysis_plan.csv`. The manifest uses the strict future-result schema in `experiments/schemas/external_condition_result_schema.json`, assigns the condition rows to twelve 240-row shards, and records planned metrics such as routing correctness, clarification correctness, constraint compliance, parse success, latency, and token count. The dry run is an execution-readiness check; it reports no model outputs and no statistical outcomes.

`scripts/build_external_representations.py` then materializes the three representation conditions as metadata-only payload templates. It writes `results/experiments/external_representation_payloads.jsonl` and `results/experiments/external_representation_payload_index.csv`, both aligned to the dry-run shards. The original/freeform condition contains only a pinned source reference and construction instruction; the SkillOps-normalized condition exposes lifecycle fields as review-time placeholders; the ablation condition withholds the preregistered lifecycle component for that case type. This makes the external execution path concrete without copying third-party content or reporting model behavior.

Provider/model	Parse success	Expected-behavior match
DeepSeek `deepseek-v4-flash`	12/12	3/12
Kimi `kimi-k2.7-code`	4/4	2/4
Total	16/16	5/16

Table 14: Bounded external live smoke on metadata-only payload rows. These records validate the execution and summarization path; they are not powered for statistical inference and are not reported as a model ranking.

The final execution hook is `scripts/run_external_payload_experiment.py`. By default it performs only a dry run, writing `results/experiments/external_payload_run_plan.csv` and `results/experiments/external_payload_run_plan.md`. Live execution must be requested explicitly with a provider, model, shard or sample limit, and `--run-live`; unbounded live execution is refused. This is included so the study can move from protocol to bounded execution without accidentally treating the 2880-row packet as completed evidence.

For a smaller execution rehearsal, `scripts/prepare_external_pilot_plan.py` selects a seeded 24-artifact subset from within-cap external candidates: six candidate references from each of the four study-family strata. The resulting pilot has 96 base cases, 288 condition rows per provider, and 576 provider-condition rows across DeepSeek and Kimi. The selected references are still pending eligibility review; their role is to rehearse annotation, representation construction, parsing, and provider logistics, not to certify the final external corpus. `scripts/run_external_pilot_experiment.py` consumes that plan, writes a not-run provider-condition execution plan and no-secret provider-readiness table, skips rows already completed in prior pilot raw outputs, and refuses unbounded live execution. In the evidence state reported here, this pilot runner is readiness evidence; no completed pilot live slice is counted as an external effect estimate.

The human-review side of the same pilot is made concrete by `scripts/generate_external_pilot_annotation_calibration.py`. It writes a 96-case pilot annotation worklist and a balanced 32-case calibration subset: two artifacts from each study family, crossed with the four case types. These files reserve independent annotator and adjudication fields, but the fields remain empty in this release. They therefore support annotation logistics and disagreement estimation, not collected human labels.

The repository additionally includes `scripts/prepare_external_smoke_test_plan.py`, which writes a no-secret bounded smoke-test plan for DeepSeek and Kimi. In the current evidence state, the plan records selected payload ids, required environment-variable names, and the exact bounded command shape. A bounded provider smoke was then run on the first external shard prefix: 12 DeepSeek rows covering one external artifact across the four base case types and three representation conditions, and four Kimi rows covering the same artifact's first positive and negative conditions. This smoke is deliberately treated as an execution-path check, not as external validation.

`scripts/summarize_external_results.py` closes the external-evaluation loop at the analysis boundary. It reads `results/experiments/raw/external_condition_*.jsonl` files when present and writes `results/experiments/external_result_summary.csv` plus `results/experiments/external_statistical_summary.csv`. Table 14 summarizes the bounded smoke. The main signal is that the live path can produce schema-valid records without storing model prose; the low behavior-match rates are a useful negative result for the current metadata-only representations and do not support effectiveness claims.

A metadata-only static pass analyzes the Git file trees of ten GitHub-hosted sources without copying repository prose or code into the paper artifact. The pass is implemented in `scripts/anal`

Static indicator	Sources	Interpretation
README present	10/10	Public entry point or usage guide.
License file present	10/10	Reuse status can be checked at source.
Test files present	9/10	Some executable or regression-oriented structure.
Scripts or code files present	9/10	Operational logic or runnable examples.
Workflow files present	8/10	CI, automation, or workflow configuration.
Schema/config files present	9/10	Machine-readable configuration surface.
Security-related files present	7/10	Explicit security, guardrail, or policy signals.
`SKILL.md` files present	4/10	Direct skill-package indicators.

Table 15: Metadata-only static indicators across ten GitHub-hosted external sources. The counts are structural evidence for corpus diversity, not a behavioral benchmark.

`yze_external_corpus.py` and writes `results/tables/external_corpus_static_analysis.csv` plus `results/tables/external_corpus_summary.md`. Table 15 summarizes selected structural indicators. These counts show that the external frame contains lifecycle-relevant artifacts beyond the author's ecosystem, but they do not measure behavioral quality or task success.

The planned external study is designed around 240 target artifact slots across Agent Skills, MCP and tool recipes, agent workflow templates, and prompt/function recipes. Before outcome-bearing execution, the 8 pending replacement slots must be filled and all concrete references must pass eligibility review. For each eligible artifact, the protocol would construct positive-trigger, negative-trigger, boundary, and risk-or-constraint cases, yielding 960 paired cases. The planned analysis uses paired stratification, mixed-effects logistic models, McNemar robustness checks for paired binary outcomes, bootstrap intervals for F1, and Holm-Bonferroni correction across metrics. The 24-artifact pilot described above is the intended calibration step before that larger study: it is large enough to expose annotation disagreement, provider failures, parsing failures, and representation-construction mistakes, but too small to support broad external-effect claims.

The human-review layer is deliberately separated from the automated evidence. It requires 48–72 participants with agent or developer-workflow experience and measures routing success, risk discovery, review time, workload, and trust calibration. This paper does not report that human study; it reports the protocol so that the boundary between completed evidence and required external validation remains explicit.

6.7 Interpretation

Taken together, the descriptive tables document benchmark composition, the repository checks validate internal consistency, and the local security pilot shows that at least one review loop can execute end to end over risk and benign cases. The strongest supported claim is therefore not that SkillOps improves agent performance. The supported claim is narrower and, for this paper, more important: explicit skill interfaces make operational assumptions, failure modes, and review checks concrete enough to inspect, test, and revise.

These results, including the bounded external smoke, do not establish statistical significance, broad empirical validation, or general effectiveness across external skill sets, user populations, or deployment environments. The new pilot-readiness layer improves the path to such validation by making the next 24-artifact execution and annotation step explicit, but it does not change the claim boundary.

7 Discussion

SkillOps frames personal agent skills as operational artifacts rather than isolated prompts. This framing has several implications.

The main practical insight is not that SkillOps makes skill behavior deterministic. Rather, it moves trigger cases, context policies, constraints, and evidence logs into reviewable artifacts that support measurable checks and reviewable operational analysis.

First, it makes skill design easier to review. When trigger conditions, context policies, and constraints are written clearly, they can be reviewed by humans and checked by tools. This is useful even when the underlying model remains uncertain.

Second, it encourages test-driven skill development. A skill should include examples of intended activation, unintended activation, edge cases, and unsafe requests. These examples do not guarantee correct behavior, but they create a practical basis for regression testing.

Third, it treats memory as both useful and risky. Persistent memory can improve continuity across sessions, but it can also preserve outdated preferences, project state, or unsafe assumptions. SkillOps therefore treats forgetting as a first-class operation rather than an afterthought.

Fourth, it suggests that safety checks should be integrated into normal skill operation. Security scanning and self-auditing should not be reserved only for high-risk deployments. Even personal agents can perform file edits, repository operations, or external communications that require explicit safeguards.

Finally, SkillOps may help bridge individual practice and more formal agent engineering. Many agent failures are not caused solely by model limitations. They also arise from unclear instructions, uncontrolled context, weak operational boundaries, and missing review procedures. A lightweight operational framework can make these issues easier to identify and improve.

8 Limitations

This paper has several limitations.

First, the primary artifact base is authored and interpreted by one author. This paper therefore reflects a single-author artifact base rather than a multi-lab or multi-organization corpus. The added external corpus frame reduces this risk at the study-design layer, and the 24-artifact pilot worklist makes the next validation step concrete. It is still not a completed external annotation study.

Second, the artifact evidence comes from five public repositories. This is enough to motivate a framework, but it is not enough to establish generality across the broader agent ecosystem.

Third, the benchmark cases are manually constructed and exploratory. The counts reported in Section 6 describe benchmark composition rather than observed production traffic or a statistically powered experiment.

Fourth, the local security pilot is rule-based, while the DeepSeek and Kimi runs are model-backed. Agreement or disagreement between those layers should be interpreted as benchmark evidence, not as proof of deployment safety.

Fifth, the benchmark cases include manually constructed benign security controls. The reported detection and false-positive rates are therefore benchmark-specific rather than broad external validation.

Sixth, the live model results use one execution per model and a small manually constructed benchmark. They are useful for feasibility and failure analysis, but not for statistical significance or model ranking.

Seventh, skill-component ablations are supported by the harness, but broader ablation evidence requires repeated runs and independently reviewed external artifacts before strong comparative claims are warranted.

Eighth, a bounded external model smoke is reported, but no external user study is reported. The 96-case pilot worklist and 32-case calibration subset prepare two-annotator review, but they do not measure how other users understand the skill format, apply the workflow, or experience the proposed guardrails in practice.

Ninth, external skill sets, other agent environments, and independent annotations are needed before stronger comparative claims could be made. The third-party corpus frame makes that study design concrete, but does not replace running it. The pilot runner adds bounded execution mechanics and resume behavior, not completed external outcomes.

Tenth, several artifacts encode local desktop-agent assumptions about directory layout, skill packaging, cron usage, and local agent control. These assumptions may not transfer directly to hosted agents, enterprise workflows, or other agent environments.

Eleventh, SkillOps focuses on the operational layer of skill design. It does not propose a new model architecture, training method, planner, or tool-use algorithm.

Twelfth, some evaluation criteria still require human judgment. For example, audit usefulness and context relevance may be difficult to score objectively. The external annotation guide and pilot calibration files address this at the protocol level, but reliability statistics require completed independent annotations.

Thirteenth, personal agent environments differ substantially in their tool access, memory mechanisms, security policies, and user expectations. A skill format that works in one environment may require adaptation in another.

9 Related Work

Agent skills and skill-library maintenance. The recent Agent Skills ecosystem makes the skill abstraction a first-class interface rather than a private prompt convention. Anthropic's Agent Skills package procedural knowledge in file-and-folder bundles with progressive context disclosure and explicit security cautions [3]. SkillsBench evaluates curated and automatically synthesized skills across many tasks and finds that skill usefulness is large but uneven across domains, which reinforces the need to test skills as artifacts rather than assume that more procedural context is always beneficial [21]. A recent paper using the same SkillOps name studies library-level maintenance through typed skill contracts and a hierarchical skill ecosystem graph [36]. This paper is narrower in one respect and broader in another: it does not propose a library-maintenance algorithm or report environment task success, but it treats personal-agent skills as operational artifacts whose triggers, context, constraints, memory, tests, and audit rules must be inspectable even before a large skill library exists.

Tool-using and modular agents. SkillOps is discussed here mainly as a framework for agent skill artifacts within work on modular agents and tool-using language model systems, with earlier software-modularity references serving only as background analogies. Recent Agent Skills specifications and public skill repositories make this artifact layer concrete by packaging metadata, instructions, scripts, and reference materials into reusable folders [1, 4]. MCP reference servers and OpenAI Agents SDK examples provide adjacent tool and workflow ecosystems that motivate the external corpus frame [27, 29, 30]. Classical work on software decomposition and reusable structure emphasized information hiding, design patterns, and component-based modularity [12, 34, 43],

while hierarchical reinforcement learning formalized reusable temporally extended skills [11, 42]. In the LLM era, MRKL, Toolformer, and ReAct study how models call tools or coordinate reasoning with external actions [19, 38, 48]. Benchmark-oriented tool-use work such as API-Bank and ToolLLM evaluates API invocation over larger tool sets [20, 37], while SayCan and Gorilla connect language models to robotic affordances or large API spaces [17, 35]. LATM and CRAFT study tool-making or toolset customization [7, 50], and AutoGen and MetaGPT organize multi-agent software workflows [16, 46]. SkillOps differs by treating a skill as a governed artifact with trigger contracts, context policy, constraints, tests, and lifecycle metadata rather than only as an action primitive, generated tool, or coordination role.

Skill libraries and memory. Reusable skill libraries appear in embodied and long-horizon agents such as Voyager, where executable skills are stored and reused over time [45]. Generative Agents, CoALA, MemoryBank, and MemGPT show that long-running agents need explicit mechanisms for memory storage, retrieval, reflection, and control [32, 33, 41, 51]. Work on reflective improvement, including Reflexion and Self-Refine, further suggests that agent behavior can be improved by structured feedback loops [25, 40]. SkillOps extends this line by specifying which skills may write or consume memory, how staleness is handled, and how retirement or forgetting is enforced.

Agent security and prompt injection. Security work on LLM-integrated applications highlights why skill-level context policy matters. Indirect prompt injection shows that untrusted content can override intended instructions when data and control are mixed [15]. Red-teaming, formal prompt-injection benchmarking, instruction-priority training, structured-query defenses, and environments such as AgentDojo all study ways to characterize or mitigate these risks [8, 9, 13, 23, 44]. SkillOps does not claim to solve prompt injection; its narrower goal is to localize trust boundaries, permission scopes, memory-write rules, and audit fields within each skill.

Agent evaluation and software engineering. Recent agent evaluation work increasingly uses interactive or executable environments rather than static question answering. SWE-bench and SWE-agent study real software tasks and the importance of agent-computer interfaces [18, 47], while AgentBench and τ-bench broaden evaluation to multiple environments and tool-user interaction settings [22, 49]. SkillOps adopts the operational spirit of this literature but focuses on skill-level properties such as false-trigger rate, stale-context errors, constraint violations, and recovery after partial failure.

Documentation, model cards, and operational reporting. SkillOps is also informed by work on documentation and operational discipline for AI systems. Large language models and instruction tuning make natural-language control surfaces central to system behavior [6, 31], while human-AI interaction guidelines emphasize communicating system capabilities, uncertainty, and user expectations [2]. Model Cards, Datasheets, Hidden Technical Debt, and the ML Test Score argue that AI systems need structured reporting, monitoring, and readiness checks [5, 14, 26, 39]. SkillOps transfers this documentation mindset to modular agent skills by making operational assumptions explicit and reviewable.

10 Conclusion

This paper introduced SkillOps, a practical framework for designing, testing, and operating modular skills in personal AI agents. The framework is derived from open-source artifacts and emphasizes explicit skill structure, trigger design, context injection, execution constraints, security checks, self-auditing, and forgetting. The paper formulates three research questions and reports a descriptive benchmark layer, repository-level consistency checks, a local security-guard pilot over risk and benign-control cases, and live model-backed runs over DeepSeek and Kimi. It also provides a third-party external corpus protocol, a 24-artifact pilot execution plan, bounded pilot runner readiness, and a two-annotator calibration worklist.

The contribution is bounded but operationally substantive. SkillOps is not presented as a new agent architecture or as a broadly validated empirical result. Instead, it identifies an underspecified layer in personal-agent systems: the lifecycle discipline required to make reusable skills inspectable, testable, and maintainable. The controlled pilots show that structured skill artifacts can support executable checks, live model-backed review loops, and reviewable operational analysis in this setting. The external protocol and pilot-readiness artifacts make the next validation step auditable, while broader empirical claims still require external skill sets, repeated execution studies, independent annotation, and external use.

References

[1] Agent Skills. Agent skills specification. `https://agentskills.io/specification`, 2026. Accessed 2026-06-25.

[2] Saleema Amershi, Daniel S. Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi T. Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for human-ai interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233. URL `https://doi.org/10.1145/3290605.3300233`.

[3] Anthropic. Equipping agents for the real world with agent skills. `https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills`, 2025. Published October 16, 2025.

[4] Anthropic. Public repository for agent skills. `https://github.com/anthropics/skills`, 2026. Accessed 2026-06-25.

[5] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ml test score: A rubric for ml production readiness and technical debt reduction. In *2017 IEEE International Conference on Big Data*, 2017. URL `https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/`.

[6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL

https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[7] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=qV83K9d5WB.

[8] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. Struq: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*, 2025. URL https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe.

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=m1YYAQjO3w.

[10] DeepSeek. Deepseek api documentation. https://api-docs.deepseek.com/, 2026. Accessed 2026-06-25.

[11] Thomas G. Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. URL https://www.jair.org/index.php/jair/article/view/10266.

[12] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN 9780201633610. URL https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480.

[13] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022. URL https://arxiv.org/abs/2209.07858.

[14] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

[15] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023. doi: 10.1145/3605764.3623985.

[16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

[17] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the Conference on Robot Learning*, 2023. URL https://proceedings.mlr.press/v205/ichter23a.html.

[18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[19] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholtz. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022. URL https://arxiv.org/abs/2205.00445.

[20] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

[21] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. URL https://arxiv.org/abs/2602.12670.

[22] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

[23] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, 2024. URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei.

[24] Song Luo. Skillops paper and artifacts. https://github.com/rrrrrredy/skillops-paper/releases/tag/v1.2.0, 2026.

[25] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=S37hOerQLB.

[26] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.

[27] Model Context Protocol. Model context protocol servers. https://github.com/modelcontextprotocol/servers, 2026. Accessed 2026-06-25.

[28] Moonshot AI. Kimi k2.7 code api documentation. https://platform.kimi.ai/docs/guide/kimi-k2-7-code-quickstart, 2026. Accessed 2026-06-25.

[29] OpenAI. Openai agents sdk javascript/typescript. https://github.com/openai/openai-agents-js, 2026. Accessed 2026-06-25.

[30] OpenAI. Openai agents sdk. https://github.com/openai/openai-agents-python, 2026. Accessed 2026-06-25.

[31] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

[32] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[33] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. doi: 10.1145/3586183.3606763.

[34] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[35] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=tBRNC6YemY.

[36] Hongji Pu, Xinyuan Song, and Liang Zhao. Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems. *arXiv preprint arXiv:2605.13716*, 2026. URL https://arxiv.org/abs/2605.13716.

[37] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

[38] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=Yacmpz84TH.

[39] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, pages 2503–2511, 2015. URL https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.

[40] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=vAElhFcKW6.

[41] Theodore R. Sumers, Shunyu Yao, Karthik R. Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=1i6ZCvflQJ.

[42] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.

[43] Clemens Szyperski, Dominik Gruntz, and Stephan Murer. *Component Software: Beyond Object-Oriented Programming*. Addison-Wesley Professional, 2 edition, 2002. ISBN 9780201745726.

[44] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL https://arxiv.org/abs/2404.13208.

[45] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

[46] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. In *Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=BAakY1hNKS.

[47] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=mXpq6ut8J3.

[48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[49] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

[50] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=G0vdDSt9XM.

[51] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. doi: 10.1609/aaai.v38i17.29946.